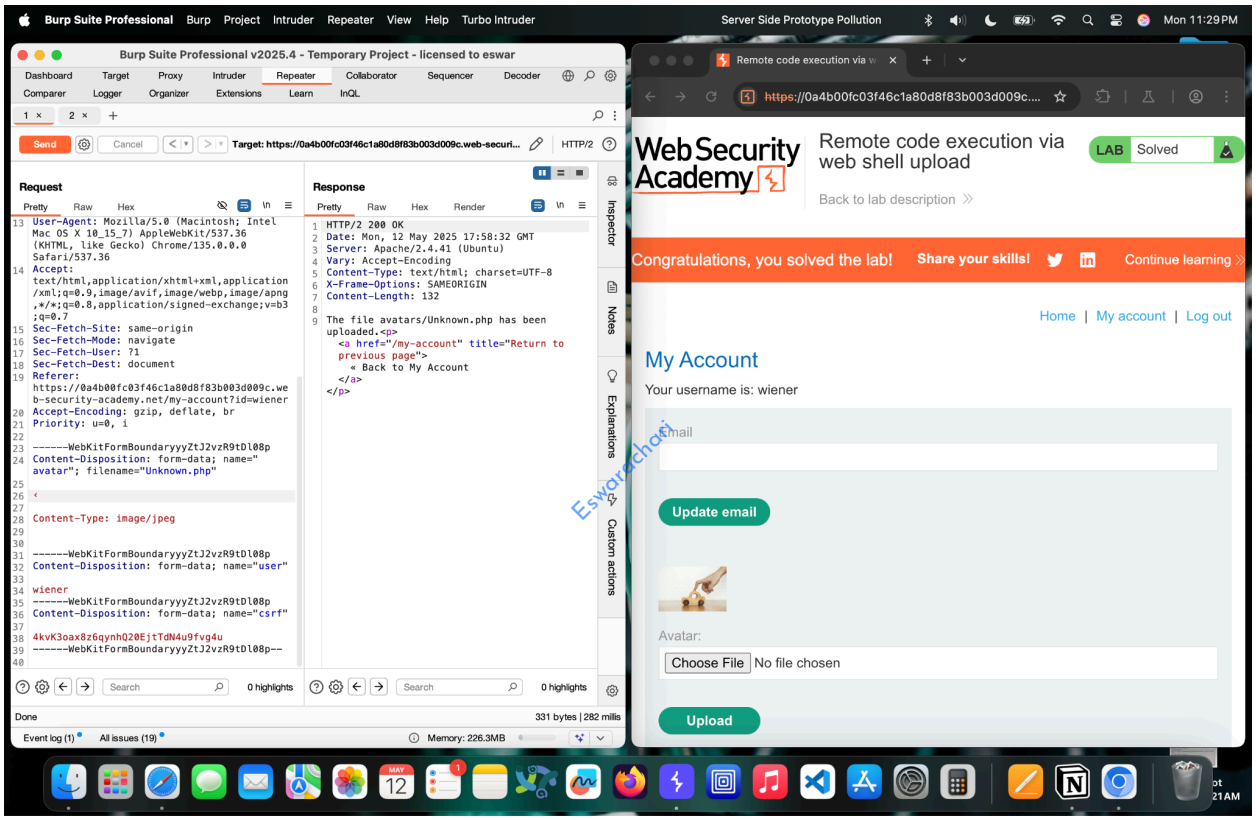
File upload vulnerabilities

1. Lab Name: Remote code execution via web shell upload

Vulnerability Type: Unrestricted File Upload / Web Shell / Remote Code Execution

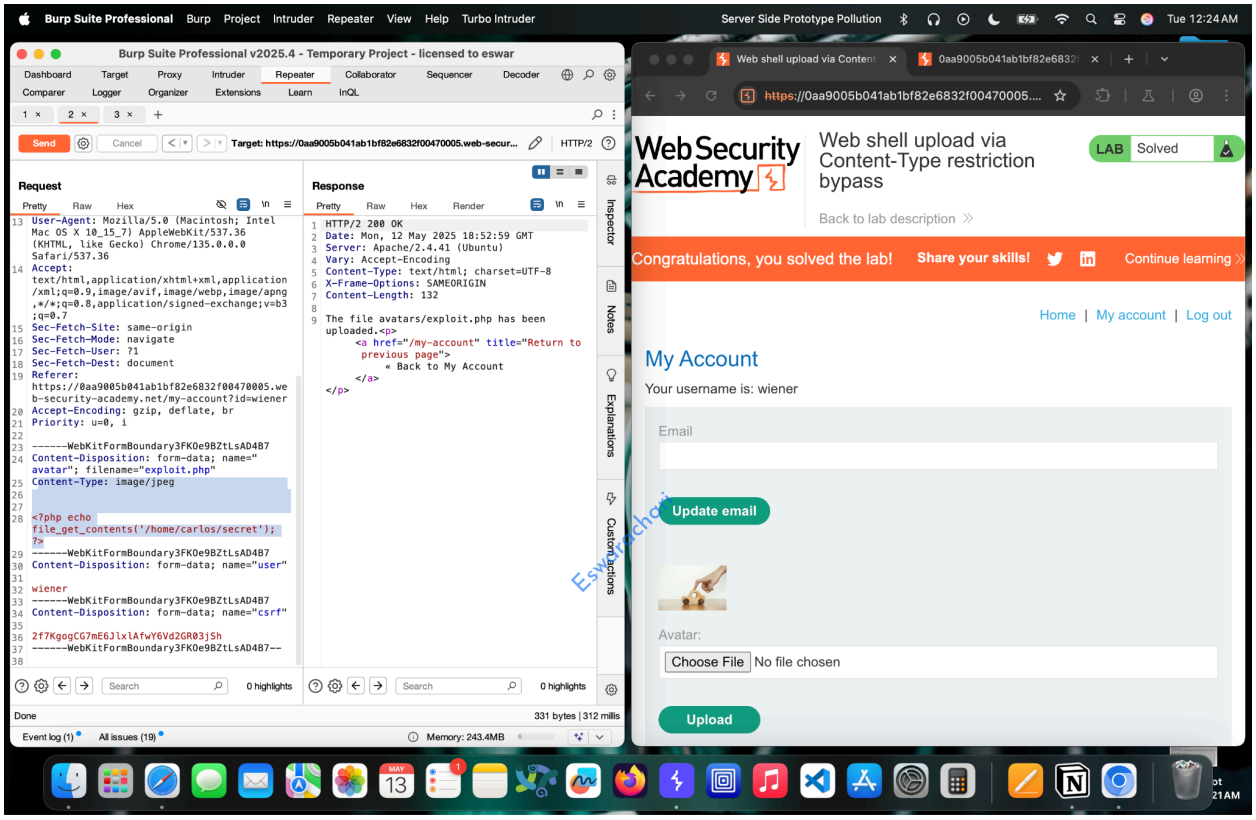


Summary: This lab demonstrates how an unrestricted file upload vulnerability can be exploited to upload a web shell. Once uploaded, the attacker can execute arbitrary operating system commands on the server, leading to remote code execution and potentially full server compromise.

Screenshot Context: The screenshot shows a Burp Suite request where a file named "unknown.php" is being uploaded. The Burp Suite interface shows the details of the file upload attempt.

2.Lab Name: Web shell upload via Content-Type restriction bypass

Vulnerability Type: Unrestricted File Upload / Content-Type Bypass / Web Shell

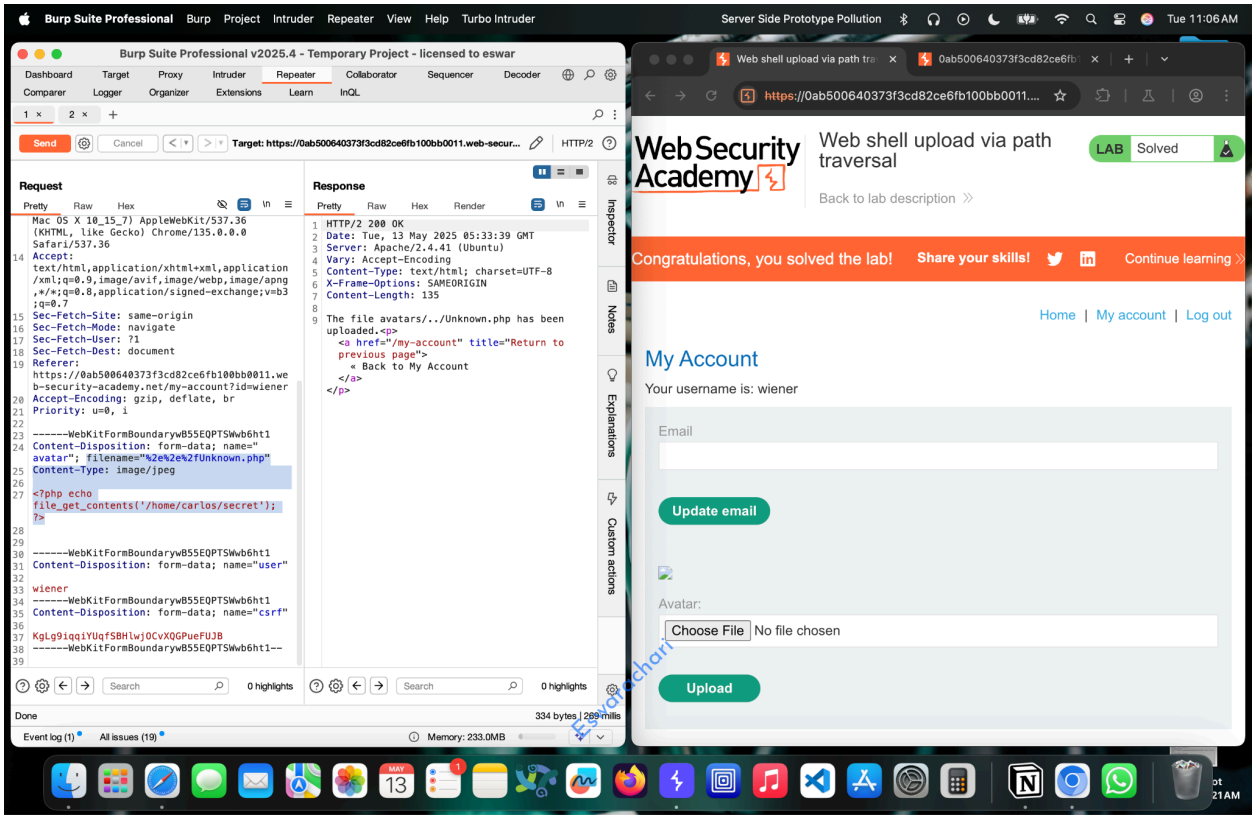


Summary: This lab demonstrates bypassing `Content-Type` restrictions to upload a web shell. By manipulating the `Content-Type` header in the HTTP request, the attacker can upload a file with a disallowed extension (e.g., "exploit.php") as if it were an allowed type (e.g., `image/jpeg`). This allows for the execution of arbitrary code on the server.

Screenshot Context: The screenshot shows a Burp Suite request with the `Content-Type` header modified, attempting to upload "exploit.php." The Burp Suite interface indicates manipulation of the request to bypass content type checks.

3.Lab Name: Web shell upload via path traversal

Vulnerability Type: Unrestricted File Upload / Path Traversal / Web Shell

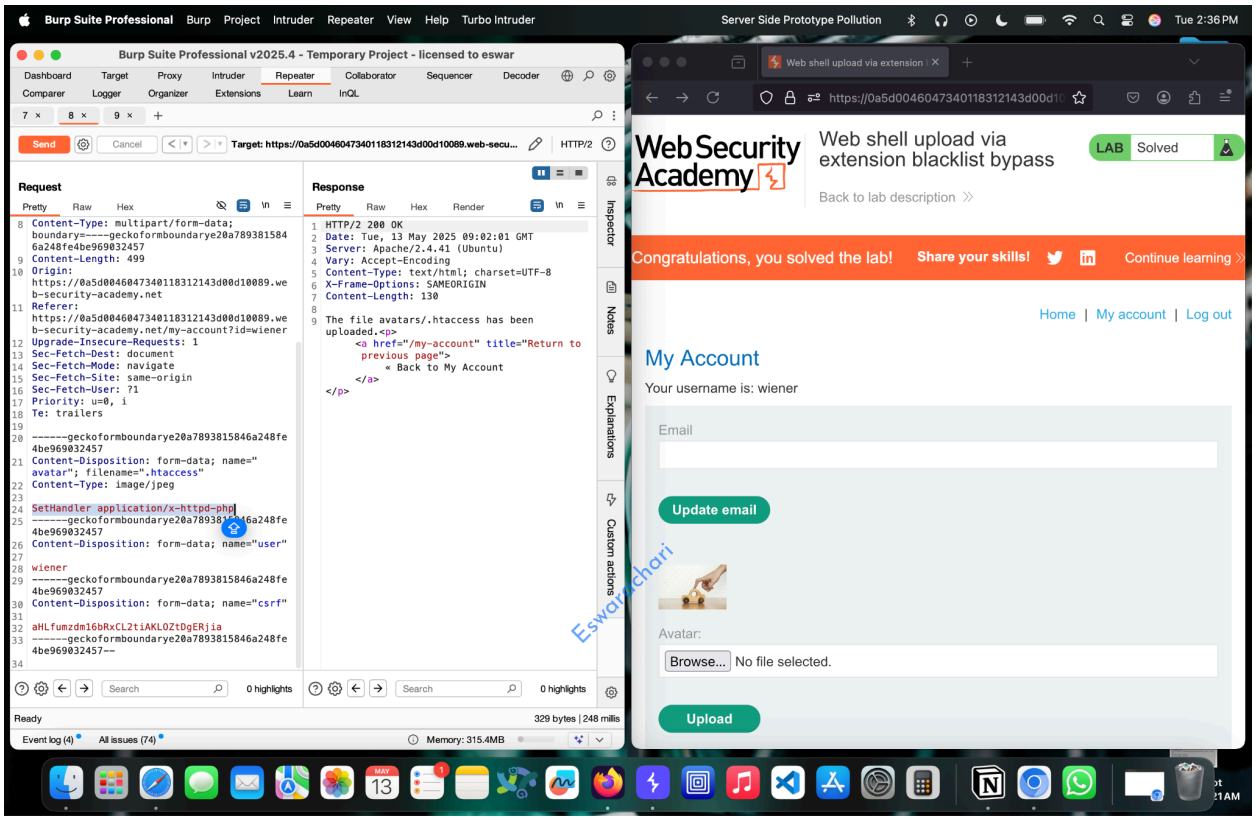


Summary: This lab demonstrates how a path traversal vulnerability in the file upload mechanism can be used to upload a web shell to an arbitrary location on the server. By manipulating the filename or path during the upload, the attacker can bypass intended upload directories and place the web shell in a publicly accessible location.

Screenshot Context: The screenshot shows a Burp Suite request attempting to upload a file named ".../unknown.php," indicating an attempt to traverse directories. The Burp Suite interface shows the file path manipulation.

4.Lab Name: Web shell upload via extension blacklist bypass

Vulnerability Type: Unrestricted File Upload / Extension Blacklist Bypass / Web Shell

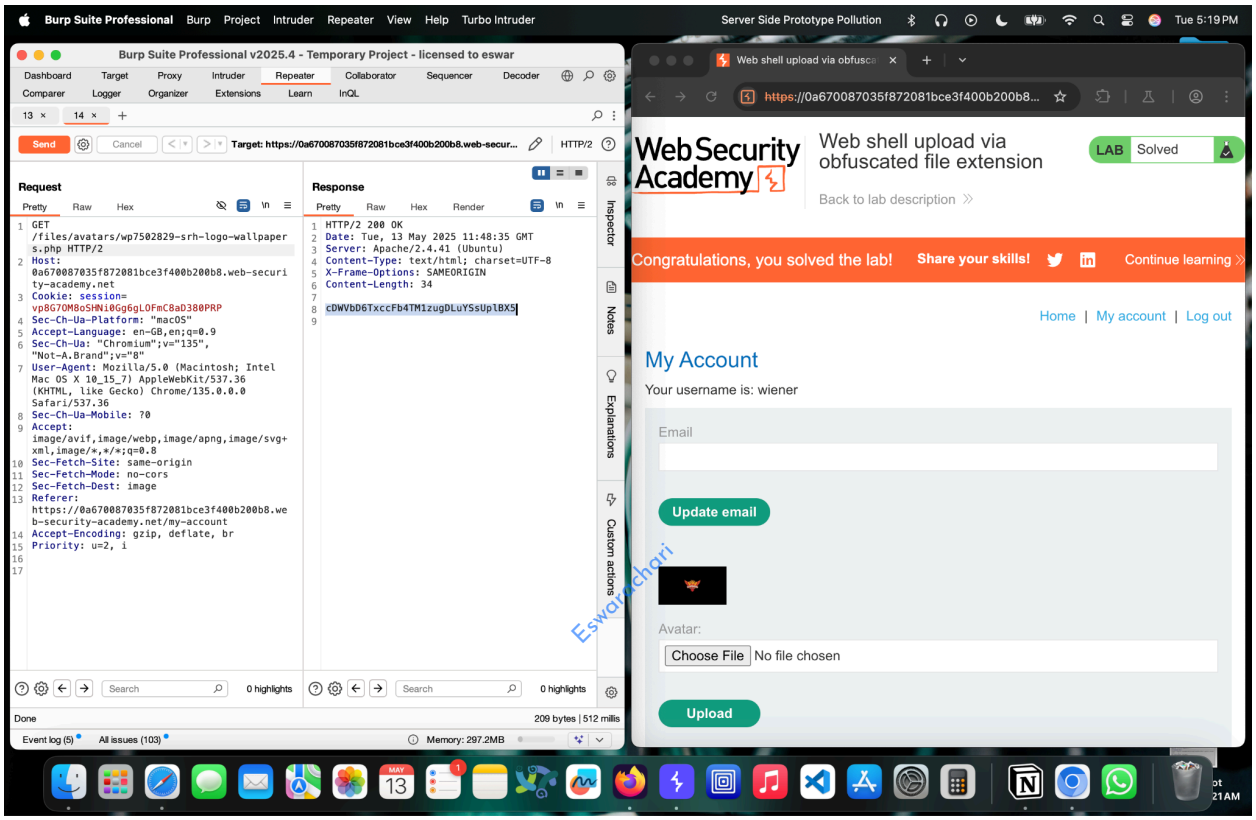


Summary: This lab shows how to bypass a blacklist of disallowed file extensions to upload a web shell. By using a different or manipulated file extension that is not on the blacklist (e.g., "hthaccess" or a mixed-case extension), the attacker can upload and execute malicious code on the server.

Screenshot Context: The screenshot shows a Burp Suite request uploading a file named "htaccess." The Burp Suite interface shows the details of the file upload attempt.

5.Lab Name: Web shell upload via obfuscated file extension

Vulnerability Type: Unrestricted File Upload / Extension Obfuscation / Web Shell

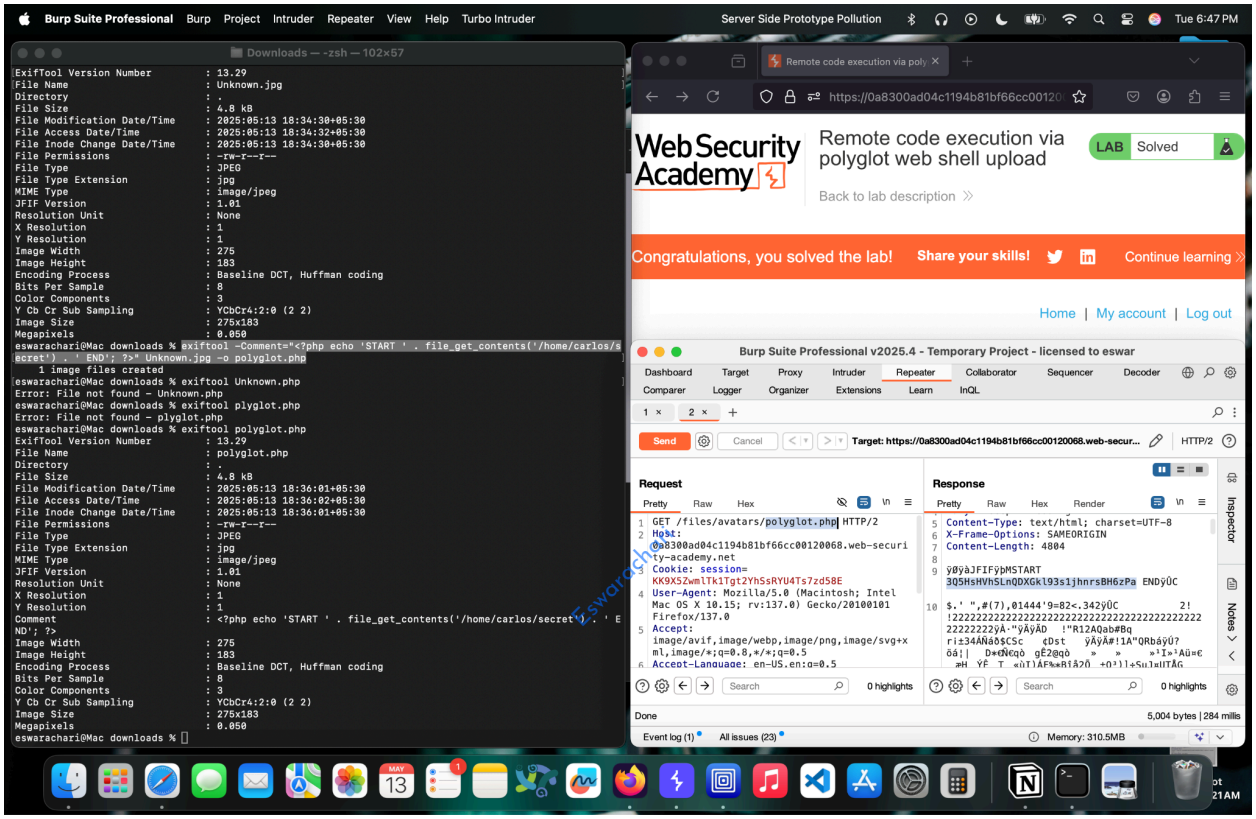


Summary: This lab demonstrates bypassing file extension restrictions by using an obfuscated or unusual file extension that the server-side validation might not recognize or block. This could involve adding special characters, multiple extensions, or URL encoding to the filename.

Screenshot Context: The screenshot shows a Burp Suite request uploading a file with an obfuscated extension ("_.php"). The Burp Suite interface shows the file name and other parameters.

6.Lab Name: Remote code execution via polyglot web shell upload

Vulnerability Type: Unrestricted File Upload / Polyglot Web Shell / Remote Code Execution



Summary: This advanced lab involves uploading a polyglot web shell, a file that is valid in multiple formats (e.g., an image and PHP). By crafting such a file, the attacker can bypass multiple layers of security checks and execute arbitrary code on the server by accessing the file as a script.

Screenshot Context: The screenshot shows a split view with Burp Suite displaying the upload request and a terminal displaying command execution (`file_get_contents('/home/carlos/secret')`) via the uploaded web shell ("polyglot.php"). The Burp Suite and terminal outputs confirm the successful exploitation.

File upload vulnerabilities : Key Topics Explored



Eswarachari

Preventing file execution in user-accessible directories 3 of 3

- Preventing file execution in user-accessible directories
- Preventing file execution in user-accessible directories - Continued
- Lab: Web shell upload via path traversal PRACTITIONER

Insufficient blacklisting of dangerous file types 7 of 7

- Insufficient blacklisting of dangerous file types
- Overriding the server configuration
- Overriding the server configuration - Continued
- Lab: Web shell upload via extension blacklist bypass PRACTITIONER
- Obfuscating file extensions
- Obfuscating file extensions - Continued
- Lab: Web shell upload via obfuscated file extension PRACTITIONER

Flawed validation of the file's contents 3 of 3

- Flawed validation of the file's contents
- Flawed validation of the file's contents - Continued
- Lab: Remote code execution via polyglot web shell upload PRACTITIONER

Exploiting file upload race conditions 4 of 4

Eswarachari

- Exploiting file upload race conditions
- Exploiting file upload race conditions - Continued
- Race conditions in URL-based file uploads
- Race conditions in URL-based file uploads - Continued

Exploiting file upload vulnerabilities without remote code execution 3 of 3

- Exploiting file upload vulnerabilities without remote code execution
- Uploading malicious client-side scripts
- Exploiting vulnerabilities in the parsing of uploaded files

Uploading files using PUT 1 of 1

- Uploading files using PUT

How to prevent file upload vulnerabilities 1 of 1

- How to prevent file upload vulnerabilities